11주차 배포한 모델은 어떻게 낡고, 언제 다시 배워야 할까

> **11주차 · 모니터링**
>
> 이 장의 수치는 모두 `practice/chapter11/`의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다.
> 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.
>
> 다른 주차의 산출물을 가져다 쓴 곳에는 그 주차의 `practice/chapter{N}/data/output/` 경로를 함께 적었다.

☐ 10주차에서 검증을 통과한 모델 이미지가 서버에서 운영 중이라고 가정함
  ○ 배포는 모델 운영의 끝이 아니라 시작임
  ○ 코드가 한 줄도 바뀌지 않아도 세상이 바뀌면 예측이 오류 없이 어긋남
    - 그 어긋남은 서버 로그에 오류로 남지 않음
☐ 이 장은 그 어긋남을 무엇으로 재고, 신호가 떴을 때 무엇을 확인한 뒤 다시 배울지 정하는 절차를 다룬다
☐ 이 장도 설명과 실습을 나누지 않음
  ○ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐
  ○ 순서대로 실행하면 이 장의 수치를 직접 생성함

1. 오늘의 질문

☐ 서버가 멀쩡한데 모델만 틀리기 시작하면 무엇으로 알아차릴까?
☐ "오늘 예측이 맞았는지"를 오늘 알 수 없는 과제에서는 무엇을 감시할까?
☐ 지표 두 개가 서로 다른 답을 내면 어느 쪽을 믿어야 할까?
☐ 드리프트(drift, 분포 변화) 경보가 떴다. 바로 다시 학습해도 될까?

2. 이 장에서 실행할 것

실행 파일이 푸는 과제

☐ 이 장의 실행 파일은 한 가지 질문에 숫자로 답함: **"기준 시각의 값 묶음과 지금의 값 묶음이 서로 다른 분포인가"**
  ○ 값 묶음은 서울 40개 측정소가 같은 시각에 측정한 미세먼지(PM10, 지름 10㎛ 이하 먼지의 농도, 단위 ㎍/㎥) 값 40개임
    - 2026-06-28 22:00의 40개 값을 **기준**으로 둠
    - 비교 대상은 두 묶음임. 1시간 뒤(6/28 23:00)의 40개 값이 비교 A, 9일 뒤 같은 시각(7/7 22:00)의 40개 값이 비교 B임

○ 세 묶음은 1주차에서 쓴 에어코리아 실시간 측정 API로 받아 `practice/chapter11/data/input/`에 JSON 파일로 저장해 두었음

□ 실행 파일은 두 묶음을 받아 여섯 단계를 차례로 출력함

단계	하는 일	출력에서 볼 값
[1] 입력	두 파일을 읽고 측정소별 값을 짝지음	측정소 수, 평균, 오른 곳·내린 곳 수
[2] 구간 나누기	기준 값으로 경계를 정해 5구간으로 나누고 구간마다 측정소 수를 셈	구간별 측정소 수와 비율
[3] KS 검정	두 묶음의 차이를 우연으로 보기 어려운지 확률로 계산	D, p
[4] PSI	구간마다 비율이 얼마나 옮겨 갔는지 더함	구간별 기여값과 합계
[5] 등급 판정	[3]과 [4]의 결과를 규칙에 넣어 정상·주의·경보 중 하나로 정함	조건식과 등급
[6] 권고	등급에 따라 사람이 할 일을 적음	권고 문장

표 11.1 실행 파일이 출력하는 여섯 단계

□ 이 과제가 모델 감시와 같은 절차인 이유

○ 배포한 모델에는 매일 새 입력이 들어옴. 입력 값의 분포가 학습할 때와 달라지면 예측이 틀리기 시작할 수 있음

○ 그 변화는 정답을 기다리지 않고 입력 값만으로 확인할 수 있음. 학습 시점 입력을 기준 묶음, 오늘 입력을 현재 묶음으로 넣으면 위 여섯 단계가 그대로 쓰임

○ 이 장은 모델 입력 대신 미세먼지 값을 넣어 절차를 실행함. 그러므로 여기서 나온 등급은 대기질 값 분포에 대한 판정이며, 9·10주차 민원 예측 모델의 상태로 읽으면 안 됨

□ 배포된 모델을 감시하는 일은 다섯 항목으로 나뉨

감시 항목	무엇을 재고 언제 알 수 있는가	다루는 절
시스템 지표 — 지연·처리량·오류율	서비스가 응답하는가. 요청 단위로 즉시 나옴	3
모델 성능 — 예측과 정답의 오차	예측이 맞았는가. 정답이 도착해야 확정됨	3
입력 분포 — KS 검정과 PSI	모델이 받는 값의 분포가 변했는가. 정답 없이 지금 잴 수 있음	4·5
두 지표의 조합 규칙	KS와 PSI가 엇갈릴 때 어느 등급으로 확정하는가	5
원인 분류와 재학습 판단표	경보가 떴을 때 무엇을 확인한 뒤 재학습하는가	6

표 11.2 이 장의 다섯 감시 항목

□ 마지막 두 항목이 이 장의 결론에 해당함

○ 경보에서 조치까지는 한 걸음이 아니라 두 걸음이고, 그사이에 원인 확인이 들어감

□ 감시를 모델 안에 두지 않고 모델 밖에서 따로 계산하는 이유가 있음

○ 모델은 틀린 답을 내면서도 `200 OK`로 정상 응답하므로, 틀렸다는 사실이 응답에는 나타나지 않음

○ 그래서 입력 분포를 서빙 밖에서 따로 계산해 기준 시점과 비교함

○ 그렇게 계산한 두 지표가 서로 다른 답을 내는 경우가 5절에서 숫자로 나타남

□ 실행하기 전에 네 가지를 먼저 적음

○ 산출물을 먼저 열면 거기 적힌 값이 정답으로 보여 "왜 이 값인가"를 묻지 않게 됨

표 11.3 예측표 — 실행하기 **전에** 채운다

예측할 것	내 예측	실제	어긋났다면 원인
같은 날 1시간 간격 비교에서 PSI가 주의 임계 0.1을 넘는가			
9일 간격 비교에서 KS 검정이 유의(p<0.05)하게 나오는가			
9일 간격 비교의 알림 등급은 정상·주의·경보 중 무엇인가			
9일 뒤 비교에서 40개 측정소 중 몇 곳이 최하 구간(17 미만)에 들어가는가			

□ 이 장에서 실행할 코드는 둘이고 하나는 선택임

순서	실행할 파일	하는 일	절	선행 조건
1	`run_chapter11.py`	비교 A·B를 차례로 돌리고 보고서를 만듦	4·5·6	`code/requirements.txt` 설치
2	`code/11-1-drift-monitoring.py`	기준·현재 한 쌍만 비교 (기본값이 비교 B)	5	위와 같음
선택	`code/11-0-collect-snapshot.py`	스냅샷(snapshot)을 새로 수집	—	data.go.kr 인증키

○ 1번이 2번을 두 번 호출하므로 1번만 실행해도 이 장의 산출물이 전부 생김

○ Docker나 서버가 필요 없고 Python과 numpy·pandas·scipy만 있으면 됨

```
cd practice/chapter11
python3 -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r code/requirements.txt
```

- **선택 실습인 `11-0-collect-snapshot.py` 는 인증키가 없으면 실행하지 않는다**
- 이 스크립트는 `data/input/` 의 스냅샷 파일을 덮어쓴다. 저장소에 커밋된 세 스냅샷이 이 장의 기준선이므로, 덮어쓰면 본문의 수치와 대조할 수 없게 된다.
- 1주차에서 만든 인증키가 있고 수집 절차를 직접 보고 싶을 때만 `--out` 을 새 파일명으로 지정해 실행한다.

3. 감시는 두 층이다 — 살아 있는가와 옳은 답인가

□ 모델 감시를 이야기하기 전에 순서를 지켜야 함

○ 예측이 틀리는 것보다 먼저, 그리고 훨씬 자주, 서비스 자체가 느려지거나 멈춤

○ 응답하지 않는 API 앞에서 예측 품질은 논점이 아님

□ 그래서 감시의 1층은 모델과 무관한 세 질문으로 시작함

○ 요청이 얼마나 걸리는가(지연)

○ 얼마나 많이 오는가(처리량)

○ 얼마나 실패하는가(오류율)

□ 지연은 평균으로 보면 안 됨

○ 4주차 실측에서 200건을 보낸 처리 지연은 평균 3.5ms인데 최대는 110.7ms였음

○ 평균만 보면 "3.5ms짜리 서비스"지만 가장 오래 기다린 요청은 그 30배 넘게 기다린 셈임

- 그래서 운영 감시는 평균 대신 분위수(p50·p95·p99)를 나란히 봄

□ 처리량은 급증만 신호가 아니라 급락도 신호임

○ 민원 접수가 평소의 10분의 1로 떨어졌다면 민원이 줄어든 것이 아니라 상류 수집이 멈췄을 가능성을 먼저 의심함

□ 오류율은 4xx와 5xx를 나눠 셈

○ 10주차에서 잘못된 입력이 422로 거절된 것은 서비스의 실패가 아니라 입력 계약이 정상 작동한 결과임

- 그러므로 오류율 0이 목표가 아님

○ 다만 422가 갑자기 급증하면 호출하는 쪽이 일제히 틀리기 시작했다는 뜻이므로 상류의 스키마 변경(2주차)을 의심함

```
# 출처: practice/chapter4/data/output/ch4_delivery_report.json
"latency_ms": { "count": 200, "mean": 3.5, "max": 110.7 }
```

```
# 출처: practice/chapter10/data/output/ch10_deploy_report.json
"host": { "forecast": 6.0, "predict_status": 200, "invalid_input_status": 422 },
"shadow_observation": { "x_prev_count": 9, "champion_forecast": 6.0,
                        "challenger_forecast": 7.3636 }
```

□ 화면에 나온 값에서 읽을 것

○ **평균과 최대가 30배 넘게 차이 남** — 지연을 한 숫자로 요약하면 가장 오래 기다린 이용자가 통계에서 사라짐

○ **정상 응답 200과 거절 422가 함께 정상임** — 422는 계약에 맞지 않는 입력을 서비스가 걸러 낸 결과이므로, 감시 규칙은 "422 발생"이 아니라 "422 급증"을 봄

○ **champion과 challenger가 같은 입력에 다른 답을 냄** — 전일 9건에 champion은 6.0, challenger는 7.3636을 냄. 두 값의 격차 자체가 모델 층에서 볼 수 있는 신호임

2층이 필요한 이유 — 오류 없이 틀리는 고장

□ 문제는 이 세 지표가 전부 정상인 채로 모델이 틀리기 시작하는 상황임

○ 서버는 빠르고 오류율은 정상이고 `/health`는 ok인데 예측만 세상과 어긋남

○ 1층 감시로는 이 고장이 잡히지 않으므로 "옳은 답을 내는가"를 묻는 2층을 따로 세움

□ 2층에서 가장 정확한 지표는 성능임

○ 예측과 정답을 대조해 오차를 계산하면 됨

○ 그런데 정답이 늦게 옴

- 민원 건수 예측의 정답은 다음 날이 되어야 확정되므로 오늘의 성능을 오늘 알 수 없음
- 심사 결과나 재방문 여부처럼 며칠에서 몇 주가 걸리는 과제도 흔함

□ 이 **레이블 지연** 때문에 모델 감시는 정답이 도착하는 순서대로 층을 이룸

○ 정답 없이 지금 볼 수 있는 것은 입력 분포와 예측 분포임

○ 정답이 도착하면 지연된 성능 평가로 확정 판정을 내림

○ 그래서 감시 설계의 첫 질문은 "어떤 지표를 볼까"가 아니라 **"이 과제의 정답은 언제 도착하는가"**다

- 정답 확정에 몇 달이 걸리는 과제에서는 입력 분포가 사실상 유일하게 볼 수 있는 신호가 됨

□ 입력 분포와 예측 분포 중 어느 쪽을 볼지는 모델이 정함

○ 9주차에서 champion으로 승격된 모델은 학습 자료의 평균을 그대로 답하는 평균 예측기임

- `serving_via_alias`가 전일 9건 입력에 6.0을 답했고 (`practice/chapter9/data/output/ch9_experiment_report.json`), 10주차의 컨테이너와 호스트도 같은 6.0을 냈음

○ 입력이 무엇이든 같은 값을 내는 모델은 입력 분포가 아무리 변해도 예측 분포가 상수로 남음

○ **입력에 둔감한 모델일수록 예측 분포가 아니라 입력 분포를 직접 감시해야 한다.**

- 이 장의 실습이 하는 일이 그것임

4. 무엇과 무엇을 비교할지 먼저 정하고 실행한다

□ 모델은 학습 당시의 데이터가 담고 있던 세상을 전제로 동작함

○ 그 전제가 깨지는 방식이 둘이고, 둘은 감지 재료부터 다름

□ **데이터 드리프트**는 입력의 분포가 변하는 것이다

○ 휴가철이 되어 지역구별 민원 건수 자체가 커지면 모델이 받는 질문의 분포가 변한 것이며, 입력과 정답의 관계는 그대로일 수도 있음

○ 입력 로그만 있으면 감지되므로 정답을 기다릴 필요가 없음

□ **컨셉 드리프트**는 입력과 정답의 관계가 변하는 것이다

○ 새 제도가 시행되어 같은 "전일 9건"에서 기대되는 다음 날 건수가 달라지면, 질문은 같은데 정답이 변한 것임

○ 관계의 변화이므로 정답을 확보해야 확인되고, 3절의 레이블 지연이 그대로 병목이 됨

□ 이 비대칭에서 두 가지 함정이 나옴

○ 입력 분포가 그대로여도 관계는 변할 수 있음 — 입력만 감시하면 이 변화가 드러나지 않음

○ 반대로 입력 분포가 변해도 성능은 멀쩡할 수 있음 — 모델이 둔감한 방향의 변화라면 그러함

□ 두 함정을 합치면 이 장에서 가장 중요한 설계 원칙이 나옴

○ **드리프트 경보는 "성능이 떨어졌다"는 확정이 아니라 "확인에 착수하라"는 신호로 설계한다.**

○ 6절의 판단표가 이 원칙을 절차로 옮긴 것임

□ 분포를 비교하기 전에 먼저 정할 것이 있음 — **무엇과 무엇을 비교할 것인가**

○ 실습은 2절에서 설명한 세 묶음을 씀: 기준 2026-06-28 22:00, 비교 A 같은 날 23:00, 비교 B 9일 뒤 2026-07-07 22:00

○ 이 자료로 확인할 수 있는 것은 데이터 드리프트뿐임

- 스냅샷에는 측정값만 있고 "정답" 열이 없으므로 관계의 변화는 이 실습에서 확인하지 못함
- 컨셉 드리프트를 확인하려면 예측과 실제를 짝지은 기록이 쌓여야 하고, 그 기록이 쌓이는 속도가 곧 레이블 지연임

○ 비교 B를 기준과 **같은 22:00**으로 고른 것은 의도한 설계임

- PM10은 하루 안에서도 오르내리므로 기준 22:00과 현재 14:00을 비교하면 "9일 동안 생긴 변화"와 "시각 차이로 생긴 변화"가 섞임
- 그러면 값이 달라져도 둘 중 무엇이 원인인지 말할 수 없음

○ 기준 묶음과 비교 묶음을 같은 조건(같은 시각, 같은 측정소 범위)으로 맞추는 것이 드리프트 감시의 첫 단계임

□ 실습을 실행함. 4~6절의 숫자는 전부 이 명령과 5절 명령의 출력임

```
cd practice/chapter11
python run_chapter11.py
```

□ `run_chapter11.py`는 실행 파일 `code/11-1-drift-monitoring.py`를 두 번 부르고 결과를 묶음

○ 화면에 나오는 순서는 다음과 같음

```
1. `=== 비교 A ===` 머리줄 다음에 비교 A의 [1]~[6]
2. `2장 실측 재현: True` — 비교 A의 KS 값이 2주차와 같은지 확인한 결과
3. `=== 비교 B ===` 머리줄 다음에 비교 B의 [1]~[6]
4. 보고서 파일 경로와 마지막 줄 `CH11_RUN_PASS`
```

○ 이 절은 비교 A와 마지막 줄을 보고, 5절에서 비교 B를 [1]부터 [6]까지 하나씩 읽음

비교 A — 1시간 간격

```
# 출처: practice/chapter11/results/run_chapter11.log (비교 A의 [1]·[2])
=== 비교 A: 22:00 vs 23:00 (6/28, 2장 재현 검증) ===
[1] 입력 — 서울 측정소의 pm10Value 값 두 묶음
    기준 airquality_seoul_2200_0628.json  시각 2026-06-28 22:00  측정소 40곳  평균 20.025
    현재 airquality_seoul_2300_0628.json  시각 2026-06-28 23:00  측정소 40곳  평균 20.775
    두 시각에 모두 있는 측정소 40곳: 오름 21곳, 내림 14곳, 같음 5곳
    예(가나다순 앞 3곳): 강남구 21→18, 강남대로 17→23, 강동구 27→22

[2] 구간 나누기 — 기준 값의 분위수로 경계를 정해 5구간으로 나눔
    구간             기준(곳)  현재(곳)  기준 비율  현재 비율
    [-inf, 17.0)        7         3     0.1765     0.0824
    [17.0, 19.0)        5         6     0.1294     0.1529
    [19.0, 21.0)       10        12     0.2471     0.2941
    [21.0, 22.0)        5         3     0.1294     0.0824
    [22.0, inf)        13        16     0.3176     0.3882
```

□ 비교 A의 [1]·[2]에서 읽을 것

○ **1시간 사이에 측정소 40곳 중 35곳의 값이 바뀜** — 21곳이 오르고 14곳이 내렸으며, 평균은 20.025에서 20.775로 0.75 올랐음

○ **구간별 측정소 수는 많아야 4곳 바뀜** — 최하 구간(17 미만)이 7곳에서 3곳으로 4곳 줄었고, 나머지 구간은 1~3곳씩 바뀌었음

```
# 출처: practice/chapter11/results/run_chapter11.log (비교 A의 [3]~[5]와 재현 확인)
[3] KS 검정 — 이 차이를 우연으로 보기 어려운가
    D = 0.175  (두 누적분포가 가장 멀리 벌어진 간격)
    p = 0.5786  → 유의수준 0.05보다 크다 → 유의하지 않음

[4] PSI — 구간마다 (현재 비율 − 기준 비율) × ln(현재 비율 ÷ 기준 비율)을 더함
    ...
    합계 PSI = 0.1193  (관행 임계: 0.1 이상 주의, 0.25 이상 경보 → 주의 대역)
    ...
[5] 등급 판정 — 경보 = KS 유의 AND PSI≥0.25, 주의 = KS 유의 OR PSI≥0.1
    ...
    등급 = 주의
...
2장 실측 재현: True
```

□ 비교 A의 [3]~[5]에서 읽을 것

○ **`2장 실측 재현: True`를 먼저 확인함** — 같은 두 묶음으로 2주차가 얻은 KS 0.175와 p 0.5786이 그대로 다시 나왔음(`practice/chapter2/data/output/ch2_drift_result.json`)

- 새 지표(PSI)를 추가하기 전에 옛 지표가 같은 값을 내는지 확인해 두면, 나중에 값이 달라졌을 때 원인이 자료인지 코드인지 가를 수 있음

○ **1시간 간격인데 PSI가 0.1193으로 주의 대역에 들어감** — 2주차가 "값은 변해도 분포 모양은 같다"고 판정한 두 묶음인데, PSI는 관행 임계 0.1을 넘겼음

○ 그 결과 등급이 주의로 나옴. 대기질이 1시간 만에 나빠졌다는 뜻이 아님

- 5구간에 40곳을 나누면 구간당 기준 측정소 수가 8곳 안팎임
- 그래서 측정소 몇 곳이 구간 경계를 넘기만 해도 구간 비율이 크게 바뀜. [2]에서 최하 구간 비율이 0.1765에서 0.0824로 약 9%p 바뀐 것은 측정소 4곳이 빠진 결과임
- 관행 임계를 이렇게 작은 묶음에 그대로 적용하면 평소에도 매시간 주의가 나올 수 있음

```
# 출처: practice/chapter11/results/run_chapter11.log (마지막 두 줄)
보고서 저장: data/output/ch11_drift_report.json
CH11_RUN_PASS
```

□ 마지막 줄에서 읽을 것

○ **`CH11_RUN_PASS`가 나와야 위아래 수치를 읽음** — 2주차 재현, 두 비교의 현재 시각(23:00과 7/7 22:00), 등급 값, 구간 수 5개, 보고서 파일 생성을 모두 확인했다는 뜻임

○ 하나라도 어긋나면 이 줄 대신 `CH11_RUN_FAIL(…)`이 나오고 괄호 안에 어긋난 항목이 적힘

5. 비교 B를 단계별로 읽는다 — KS와 PSI는 다른 질문에 답한다

□ 비교 B(9일 간격)만 따로 실행함. 실행 파일의 기본값이 비교 B의 두 파일이므로 인자 없이 실행해도 같은 비교가 나옴

```
cd practice/chapter11
python code/11-1-drift-monitoring.py
```

[1] 입력 — 같은 측정소의 값이 어떻게 바뀌었나

```
# 출처: practice/chapter11/results/11-1-drift-monitoring.log ([1])
[1] 입력 — 서울 측정소의 pm10Value 값 두 묶음
    기준 airquality_seoul_2200_0628.json  시각 2026-06-28 22:00  측정소 40곳  평균 20.025
    현재 airquality_seoul_2200_0707.json  시각 2026-07-07 22:00  측정소 40곳  평균 17.75
    두 시각에 모두 있는 측정소 40곳: 오름 9곳, 내림 26곳, 같음 5곳
    예(가나다순 앞 3곳): 강남구 21→21, 강남대로 17→14, 강동구 27→10
```

□ [1]에서 읽을 것

○ **파일 이름과 시각을 먼저 확인함** — 기준과 현재가 모두 22:00이므로 시각 차이로 생긴 변화는 섞이지 않았음

○ **40곳 중 26곳이 내렸음** — 오른 곳은 9곳이고, 평균은 20.025에서 17.75로 2.275 내려갔음

○ **측정소마다 바뀐 폭이 다름** — 강남구는 21 그대로이고, 강동구는 27에서 10으로 17 내려갔음

○ 평균 하나로는 "얼마나 많은 측정소가 어느 쪽으로 움직였는가"를 알 수 없음. 그래서 [2]에서 값 40개를 구간으로 나눠 셈

[2] 구간 나누기 — 기준 값으로 경계를 정한다

```
# 출처: practice/chapter11/results/11-1-drift-monitoring.log ([2])
[2] 구간 나누기 — 기준 값의 분위수로 경계를 정해 5구간으로 나눔
    구간             기준(곳)   현재(곳)   기준 비율   현재 비율
    [-inf, 17.0)        7         13      0.1765      0.3176
    [17.0, 19.0)        5          7      0.1294      0.1765
    [19.0, 21.0)       10         11      0.2471      0.2706
    [21.0, 22.0)        5          5      0.1294      0.1294
    [22.0, inf)        13          4      0.3176      0.1059
    비율 = (구간 측정소 수 + 0.5) ÷ (40 + 0.5×5) — 0곳인 구간에서 ln 계산이 막히지 않게 0.5를 더함
```

□ 구간 경계를 정하는 방법

○ 기준 값 40개를 작은 순서로 세웠을 때 하위 20%·40%·60%·80% 위치의 값(분위수)이 경계가 됨. 이 실습에서는 17, 19, 21, 22였음

○ 경계를 기준 값으로만 정하므로, 현재 값이 바뀌어도 경계는 그대로임

○ 맨 아래 구간은 `-inf`(아래 끝 없음), 맨 위 구간은 `inf`(위 끝 없음)까지 열어 둠

- 현재 값이 기준의 최솟값보다 작거나 최댓값보다 커도 버려지지 않고 양 끝 구간에 들어감. 기준 범위를 벗어난 값은 변화를 보여 주는 값이기 때문임

○ 측정값이 정수라 같은 값이 여러 개이므로 구간마다 정확히 8곳씩 나뉘지 않음. 기준 구간별 수가 7·5·10·5·13곳인 이유임

□ 비율에 0.5를 더하는 이유

○ [4]의 PSI 식에는 ln(현재 비율 ÷ 기준 비율)이 들어가는데, 어느 구간이 0곳이면 0으로 나누거나 ln(0)을 계산하게 되어 값을 구할 수 없음

○ 그래서 구간마다 0.5곳을 더한 뒤 비율을 계산함. 기준 최하 구간은 7 ÷ 40 = 0.175가 아니라 (7 + 0.5) ÷ (40 + 2.5) = 0.1765가 됨

□ [2]에서 읽을 것

○ **분포 전체가 아래로 이동했음** — 최하 구간(17 미만)이 7곳에서 13곳으로 늘었고, 최상 구간(22 이상)이 13곳에서 4곳으로 줄었음

○ **[21, 22) 구간은 5곳 그대로임** — 이 구간은 [4]에서 PSI에 0을 더함

[3] KS 검정 — 이 차이를 우연으로 보기 어려운가

```
# 출처: practice/chapter11/results/11-1-drift-monitoring.log ([3])
[3] KS 검정 — 이 차이를 우연으로 보기 어려운가
    D = 0.225  (두 누적분포가 가장 멀리 벌어진 간격)
    p = 0.2657  → 유의수준 0.05보다 크다 → 유의하지 않음
```

□ KS 2표본 검정은 2주차에서 이미 다뤘음

○ 두 묶음을 각각 작은 값부터 쌓아 "이 값 이하인 측정소의 비율"을 그린 선(누적분포)을 만들고, 두 선이 가장 멀리 벌어진 간격 D를 구함

- D = 0.225는 어떤 값 이하인 측정소 비율이 기준과 현재에서 최대 22.5%p 차이 난다는 뜻임. 40곳 기준으로 9곳임

○ p는 "두 묶음이 같은 분포에서 나왔다"고 가정했을 때 D가 이만큼 이상 나올 확률임

□ [3]에서 읽을 것

○ **p = 0.2657은 유의수준 0.05보다 큼** — 측정소 40곳만으로는 이 차이를 우연으로 보기 어렵다고 말할 근거가 부족하다는 뜻임

○ 2주차의 결론을 그대로 적용함

- p가 크다는 것은 "분포가 같다"가 아니라 "다르다고 볼 증거가 부족하다"는 뜻임
- 그리고 같은 D라도 측정소 수가 많으면 p가 작아짐. KS 결과는 묶음 크기에 따라 달라짐

[4] PSI — 구간마다 비율이 얼마나 옮겨 갔나

```
# 출처: practice/chapter11/results/11-1-drift-monitoring.log ([4])
[4] PSI — 구간마다 (현재 비율 − 기준 비율) × ln(현재 비율 ÷ 기준 비율)을 더함
    [-inf, 17.0)  (0.3176 − 0.1765) × ln(0.3176 ÷ 0.1765) = 0.0830
    [17.0, 19.0)  (0.1765 − 0.1294) × ln(0.1765 ÷ 0.1294) = 0.0146
    [19.0, 21.0)  (0.2706 − 0.2471) × ln(0.2706 ÷ 0.2471) = 0.0021
    [21.0, 22.0)  (0.1294 − 0.1294) × ln(0.1294 ÷ 0.1294) = 0.0000
    [22.0, inf)   (0.1059 − 0.3176) × ln(0.1059 ÷ 0.3176) = 0.2326
    합계 PSI = 0.3324  (관행 임계: 0.1 이상 주의, 0.25 이상 경보 → 경보 대역)
    확인: KL(기준→현재) 0.1826 + KL(현재→기준) 0.1497 = PSI 와 같은가 → True
```

☐ **PSI(Population Stability Index, 모집단 안정성 지수)**는 이 장에서 새로 배우는 지표임

○ [2]에서 만든 구간마다 아래 값을 계산해 모두 더함

```
PSI = Σ (현재 비율 − 기준 비율) × ln(현재 비율 ÷ 기준 비율)
```

○ 숫자를 넣은 예: 최상 구간은 (0.1059 − 0.3176) × ln(0.1059 ÷ 0.3176) = (−0.2117) × (−1.098) ≈ 0.2326

- 비율이 줄어든 구간은 두 항이 모두 음수, 늘어난 구간은 두 항이 모두 양수이므로 곱은 언제나 0 이상임
- 그래서 비율이 늘어난 구간과 줄어든 구간이 모두 PSI를 키움

○ 신용평가 업계에서 고객 구성이 달라졌는지 점검하려고 쓰던 지표임

- 관행 임계는 0.1을 주의, 0.25를 경보로 봄
- 이 임계에 통계적 근거가 약하다는 지적이 있음(Yurdakul 2018). **관행 임계는 출발점이며, 자료에 맞게 조정할 수 있음**

☐ [4]에서 읽을 것

○ **합계 0.3324는 경보 임계 0.25를 넘음** — PSI만 보면 경보 대역임

○ **합계의 약 70%가 최상 구간 한 곳에서 나옴** — 0.2326 ÷ 0.3324 ≈ 0.70. 22 이상이던 측정소가 13곳에서 4곳으로 줄어든 변화가 PSI의 대부분을 만들었음

○ **[21, 22) 구간의 기여는 0임** — 측정소 수가 5곳 그대로이므로 두 비율이 같음

○ 다섯 기여값을 그대로 더하면 0.3323이 되어 합계 0.3324와 0.0001 차이 남. 기여값을 넷째 자리에서 반올림해 적었기 때문이며, 합계는 반올림하기 전 값으로 계산한 것임

○ 마지막 줄의 `True`는 PSI가 KL 발산(두 분포가 얼마나 다른지를 한 방향으로 계산한 값)을 양방향으로 더한 값과 같다는 항등식을 계산으로 확인한 결과임. 이 관계는 9절 더 알아보기에서 다룸

☐ [3]과 [4]를 나란히 놓으면 두 지표가 다른 답을 냄

○ KS는 "이 차이를 우연으로 보기 어려운가"에 답하고 "어렵다고 할 근거가 부족하다"고 답했음(p = 0.2657)

○ PSI는 "구간 비율이 얼마나 옮겨 갔는가"에 답하고 "경보 수준만큼 옮겨 갔다"고 답했음(0.3324)

○ 한 문장으로 쓰면: "옮겨 간 크기는 경보 수준이지만, 측정소 40곳으로는 우연을 배제할 수 없다"

□ 두 지표는 묶음 크기에 따라 서로 반대 방향으로 틀림

○ 묶음이 크면(예: 수만 건) 아주 작은 차이도 KS에서 유의로 나옴. 이때는 PSI로 차이의 크기를 확인해야 함

○ 묶음이 작으면(이 실습의 40곳) 몇 곳만 구간을 옮겨도 PSI가 크게 나옴. 이때는 KS로 우연인지 확인해야 함

○ [5]의 규칙이 두 지표를 함께 요구하는 이유가 이것임

□ 1시간 간격(비교 A)과 비교하면, 간격이 길어지자 세 지표가 모두 커졌음

○ KS D: 0.175 → 0.225 / PSI: 0.1193 → 0.3324 / KL(기준→현재): 0.0646 → 0.1826

[5] 등급 판정 — 두 지표를 규칙에 넣는다

```
# 출처: practice/chapter11/results/11-1-drift-monitoring.log ([5])
[5] 등급 판정 — 경보 = KS 유의 AND PSI≥0.25, 주의 = KS 유의 OR PSI≥0.1
    KS 유의 = False, PSI≥0.25 = True, PSI≥0.1 = True
    경보 조건: False AND True → False
    주의 조건: False OR True → True
    등급 = 주의
```

□ 실행 파일은 등급을 아래 순서로 정함

```
# practice/chapter11/code/11-1-drift-monitoring.py 발췌 (judge_alert 함수)
if ks_significant and psi >= PSI_ALERT:      # p<0.05 그리고 PSI>=0.25
    level = "경보"
elif ks_significant or psi >= PSI_WATCH:     # 둘 중 하나만
    level = "주의"
else:
    level = "정상"
```

□ [5]에서 읽을 것

○ **경보 조건부터 검사함** — KS 유의가 False이므로 PSI가 0.25를 넘었어도 `False AND True`는 False임

○ **다음으로 주의 조건을 검사함** — PSI가 0.1 이상이므로 `False OR True`는 True이고, 등급은 주의로 정해짐

○ PSI만 보면 경보 대역이었지만, 규칙이 한 단계 낮은 주의로 정했음

□ 경보에 KS 유의와 PSI 0.25를 **AND**로 묶은 이유

○ OR로 묶었다면 둘 중 하나만 걸려도 경보가 됨. 이 실습처럼 묶음이 작으면 PSI가 0.25를 넘는 일이 잦으므로, 경보가 나갈 때마다 담당자를 부르게 됨

○ 틀린 경보를 여러 번 받은 담당자는 다음 경보를 확인하지 않게 됨. 그래서 담당자를 부르는 일은 두 지표가 모두 넘은 경우에만 함

[6] 권고 — 등급에서 바로 재학습으로 가지 않는다

```
# 출처: practice/chapter11/results/11-1-drift-monitoring.log ([6])
[6] 권고 조치
    모니터링 강화 + 원인 분류 착수(계절/정책/시스템 변경/데이터 결함)
```

□ [6]에서 읽을 것

○ **주의 등급의 권고는 "재학습"이 아니라 "원인 분류 착수"임** — 담당자를 부르지 않고 기록을 쌓으면서 왜 분포가 바뀌었는지 확인을 시작함

○ 경보 등급이었다면 권고는 "원인 분류 우선 — 데이터 결함이면 수리, 실제 분포 변화면 재학습 후보 데이터 확보 후 9장 승격 게이트로"가 됨(`recommend_action` 함수). 경보에서도 재학습을 바로 실행하지 않음

○ 원인 분류를 어떻게 하는지는 6절에서 다룸

6. 경보는 명령이 아니다 — 재학습 판단

□ 경보가 떴다고 재학습 버튼부터 누르면 안 됨

○ 알림과 조치 사이에 두 개의 확인이 들어감

□ 첫째는 **원인 분류**다

○ 2주차에서 분포 변화의 원인을 정책 변화·계절성·외부 이벤트·시스템 변경으로 나눴는데, 운영에서는 하나가 더 붙음

○ 상류 파이프라인이 깨져 단위가 바뀌었거나 결측이 급증한 **데이터 결함**임

- 이때 필요한 조치는 재학습이 아니라 수리임. 손상된 데이터로 다시 배우면 손상된 상태를 학습한 모델이 나옴

○ 원인이 계절 재발이라면 임계 보정이나 계절 피처 추가가 답임

- 매년 여름의 같은 변화를 매년 "이상"으로 경보하면 알림 피로만 쌓임

○ 정책 시행 같은 진짜 세상의 변화일 때에만 재학습이 후보가 됨

□ 둘째는 **성능 확인**이다

○ 4절에서 봤듯 입력 분포의 변화가 성능 저하를 보장하지 않음

○ 정답이 도착하는 과제라면 지연된 성능 평가로 저하를 확인한 뒤 움직임

□ 두 확인이 실제로 어떤 순서로 이루어지는지 월요일 아침 상황으로 봄

○ 대시보드에 입력 분포 경보가 떴다면 당직자는 먼저 세 곳을 확인함

- 상류 수집 로그에서 주말 배치가 정상 종료했는지 봄(데이터 결함 여부)
- 달력에서 직전이 연휴였는지 봄(계절 재발 여부)
- 정책 공지에서 새 제도가 그 주에 시행됐는지 봄

○ 성능 확인 항목은 이 아침에 채울 수 없음. 어제 실제 건수는 오늘 밤에야 확정되기 때문임

○ 그래서 아침의 조치는 재학습 결재가 아니라 원인 분류 착수와 기록 축적에 머묾

□ 실습이 만든 보고서에는 판정에 쓴 규칙과 권고가 지표와 분리되어 기록됨

```
# 출처: practice/chapter11/data/output/ch11_drift_report.json
"alert_rule": {
  "ks": "p < 0.05 (2장과 동일 alpha)",
  "psi": "0.1 주의 / 0.25 경보 (신용평가 업계 관행 — Yurdakul 2018)",
  "combine": "경보=KS유의 AND PSI>=0.25, 주의=둘 중 하나, 정상=둘 다 미달" },
"comparison_a_matches_ch2": true,
"notes": [
  "비교창 정렬: 기준·현재 모두 22:00 정시 — 일중 변동 교란을 설계로 제거",
  "n=40 소표본 — KS 검정력 낮음, PSI 구간당 기대 8건으로 출렁임에 민감",
  "알림≠재학습: 권고 조치는 원인 분류를 먼저 요구(11.4)" ]
```

□ 화면에 나온 값에서 읽을 것

○ **판정 규칙이 산출물 안에 글자로 남음** — 나중에 등급을 두고 다투면 그때 쓰인 임계와 조합 방식을 파일에서 확인할 수 있음

○ **권고가 "재학습"이 아니라 "원인 분류 착수"임** — 지표·등급·조치를 각각 다른 필드에 적어 둔 설계가 그 분리를 강제함

○ **소표본이라는 한계가 보고서에 함께 적힘** — 유리한 해석만 남기지 않고 판정의 약점을 같은 파일에 적는 것이 감사 대비의 기본임

표 11.4 재학습 트리거 판단표

드리프트 경보	성능 저하(정답 확인)	원인 분류	조치
정상	—	—	기준선 유지, 정기 감시
주의	미확인(정답 대기)	착수	감시 강화, 기록 축적 — 사람 호출 없음
경보	확인됨	데이터 결함	**수리**(재학습 금지 — 손상된 데이터를 학습하게 됨)
경보	확인됨	계절 재발	임계 보정·계절 피처 검토
경보	확인됨	실제 변화(정책 등)	**재학습** → challenger 등록 → 9주차 게이트
경보	저하 없음	실제 변화	감시 지속 — 모델이 둔감한 방향의 변화

□ 재학습을 언제 돌릴지 정하는 방식은 셋임

○ 정기 트리거는 주기마다 무조건 재학습하므로 단순하지만 변화가 없어도 비용이 듦

○ 이벤트 트리거는 판단표가 경보를 확정할 때만 움직이므로 효율적이지만 감시 장치가 선행 투자임

○ 실무의 기본값은 둘을 섞어, 정기 재학습을 기본으로 두고 경보가 뜨면 앞당겨 실행하는 방식임

□ 어느 경로든 끝은 같음

○ 재학습된 모델은 champion을 곧바로 교체하지 않고 **challenger로 등록되어 9주차의 승격 게이트를 다시 통과해야 하며**, 통과하면 10주차의 배포 절차를 다시 밟음

○ 급하다고 게이트를 건너뛰면 9주차에서 세운 감사 가능성이 그 지점에서 끊김

○ 데이터(4~7주차) → 훈련·등록(9주차) → 배포(10주차) → 감시(11주차) → 재학습 → 다시 9주차로 돌아오는 이 순환이 MLOps를 파이프라인이 아니라 루프라고 부르는 이유다

7. 핵심 요약

□ **1. 감시는 두 층이고, 2층이 없으면 오류 없이 틀리는 고장을 놓친다.**

○ 지연·처리량·오류율은 "서비스가 살아 있는가"에만 답하므로, 서버가 멀쩡한 채 예측만 틀리는 고장은 여기에 잡히지 않음

○ 지연은 평균 3.5ms와 최대 110.7ms가 한 실행 안에 함께 나타났으므로 분위수로 봄

○ 422 거절은 계약이 작동한 결과이고, 감시해야 할 것은 발생이 아니라 급증임(3절)

□ **2. 정답이 언제 도착하는가가 1차 신호를 정한다.**

○ 민원 건수 예측의 정답은 다음 날 확정되므로 오늘의 성능은 오늘 알 수 없고, 그동안 볼 수 있는 것은 입력 분포와 예측 분포임

○ 9주차 champion은 전일 9건에도 6.0을 답하는 평균 예측기라 예측 분포가 상수로 남으므로, 이런 모델은 입력 분포를 직접 감시함(3절)

□ **3. 비교창을 맞추지 않으면 나온 값이 신호가 아니라 교란이다.**

○ 비교 B를 기준과 같은 22:00으로 고정한 것은 일중 변동을 설계로 제거하려는 조치임

○ 같은 스냅샷 쌍에서 2주차 실측(KS 0.175, p 0.5786)이 재현됐음을 먼저 확인한 뒤에 새 지표를 읽음(4절)

□ **4. KS와 PSI는 다른 질문에 답하므로 함께 요구한다.**

○ KS는 "우연인가", PSI는 "얼마나 큰가"에 답함

○ 9일 비교에서 PSI 0.3324는 경보 대역인데 KS는 p=0.2657로 유의하지 않았고, AND 규칙이 이를 주의로 확정했음

○ OR로 묶었다면 소표본에서 PSI가 크게 커질 때마다 사람이 호출됐을 것임(5절)

□ **5. 경보와 재학습 사이에는 두 개의 확인이 있다.**

○ 원인이 데이터 결함이면 재학습이 아니라 수리이고, 계절 재발이면 임계 보정임

○ 정답으로 성능 저하까지 확인된 실제 변화일 때에만 재학습이 후보가 됨

○ 재학습된 모델도 challenger로 9주차 게이트를 다시 통과함(6절)

8. 과제

□ 실습을 실행해 만든 `practice/chapter11/data/output/ch11_drift_report.json`을 LMS에 업로드한다

□ 이어서 세 가지를 각각 세 문장 이내로 쓴다

1. 본인 산출물의 두 비교에서 `ks_significant`와 `psi_band`가 어떻게 엇갈렸는지 값으로 적고, 그럼에도 `alert_level`이 그렇게 정해진 과정을 `alert_rule`의 `combine` 규칙으로 재현한다.
2. `combine` 규칙이 KS와 PSI를 AND로 묶은 이유를 쓰고, OR로 묶었다면 운영에서 무엇이 달라졌을지 쓴다.
3. 드리프트 경보와 성능 저하가 함께 확인됐는데 원인이 데이터 결함으로 밝혀졌다면 왜 재학습을 실행하면 안 되는지, 그때 어떤 모델이 나오는지로 설명한다.

□ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

□ 이 장에서 자른 내용은 아래에서 이어 볼 수 있음

- ○ **PSI와 KL divergence의 관계** — 산출물의 `psi_equals_kl_sum`이 `true`인 것은 PSI가 양방향 KL의 합과 같다는 항등식을 수치로 확인한 결과임. 구간 확률과 스무딩 처리는 `practice/chapter11/code/11-1-drift-monitoring.py`의 `binned_proportions`·`psi_and_kl` 함수에 있음
- ○ **구간 수를 바꿔 보기** — `11-1-drift-monitoring.py`에 `--bins 10`을 주면 같은 자료에서 PSI가 어떻게 달라지는지 볼 수 있음. 구간 수가 다르면 PSI끼리 비교할 수 없다는 경고가 코드에 들어 있음
- ○ **스냅샷 수집 절차** — 정시 데이터가 정시보다 늦게 게시되는 문제를 폴러로 다루는 방법: `practice/chapter11/code/11-0-collect-snapshot.py`와 `practice/chapter11/data/output/ch11_collect_log.txt`
- ○ **골든 시그널의 원문** — 지연·트래픽·오류·포화 네 가지의 정의: https://sre.google/sre-book/monitoring-distributed-systems/
- ○ **KS 2표본 검정의 계산 규약** — https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html
- ○ **피처가 바뀔 때의 감시** — 입력 자체의 정의가 바뀌는 경우의 관리는 12주차, 감시까지 포함한 통합 사례는 13주차, 생성형 모델의 감시는 14주차에서 다룸